Software Architecture Evolutions

Proposition technique

RÉSEAU COCAGNE

RAZAKANIRINA Aingo
SAULNIER Simon



Contexte

Les Jardins de Cocagne (Réseau Cocagne) utilisent un outil de gestion historique qui ne répond plus complètement aux besoins actuels. La gestion des adhérents, des abonnements, des commandes et des livraisons nécessite une solution plus moderne, flexible et accessible.

Le projet consiste donc à concevoir une application web mutualisée permettant d'améliorer et de centraliser ces différents processus.

Notre objectif est de proposer une architecture technique robuste, sécurisée et évolutive capable de répondre à ces besoins tout en facilitant la maintenance et les futures évolutions du système.

Sommaire

1 Architecture logicielle

2 Langages et cadriciels

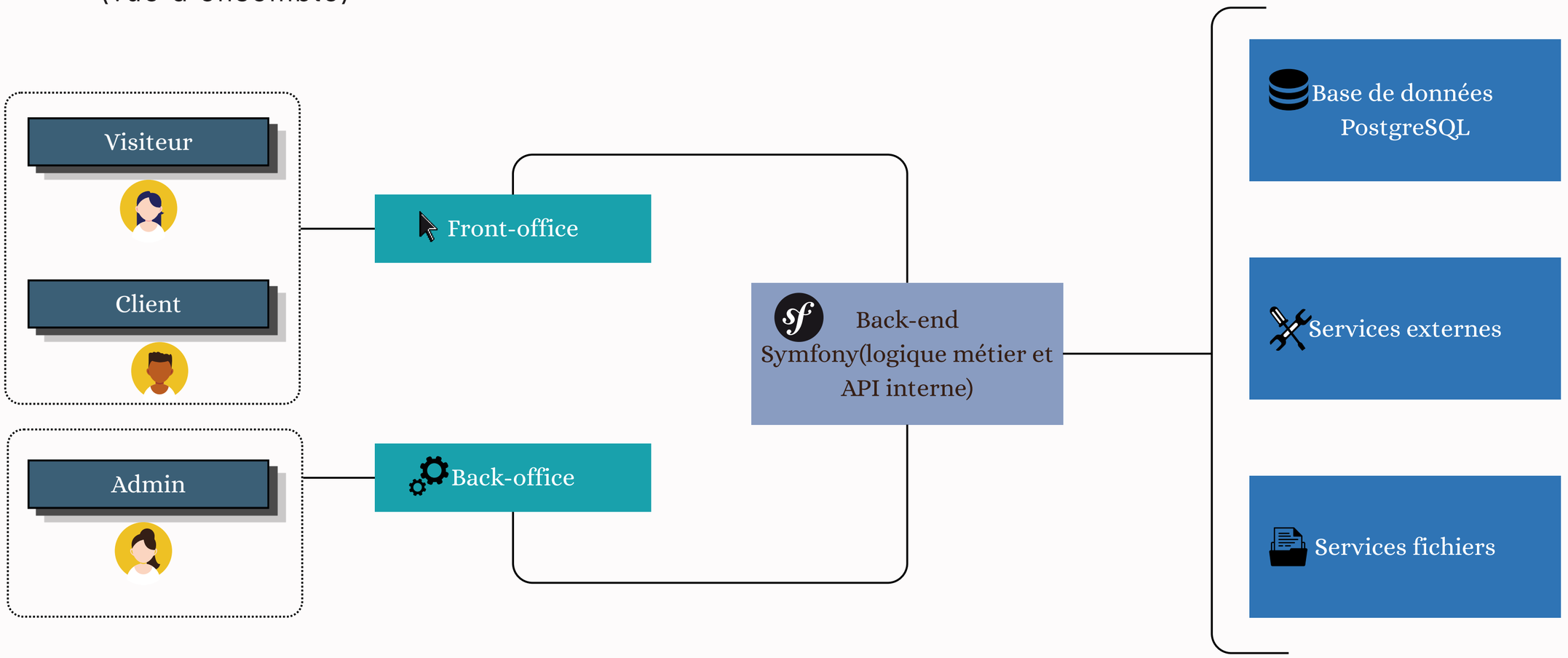
3 Base de données

4 Back office & Front office

5 Quelques solutions techniques

1. Architecture logicielle

(vue d'ensemble)



2. Langages et cadriciels

Symfony

- Architecture structurée, ce qui facilite la maintenance et l'évolution du projet.
- Gestion des routes, contrôleurs, formulaires, authentification et sécurité.
- Permet de séparer clairement les différentes parties de l'application.
- Framework mature et largement utilisé adapté au développement d'applications web.
- Facilite la prise en main des membres du jardin de cocagne sans être informaticien

PHP

- PHP est le langage sur lequel Symfony se repose.
- Langage particulièrement adapté au développement web.
- S'intègre naturellement avec PostgreSQL.
- Permet de traiter les données avant de les transmettre au navigateur.
- Écosystème important et nombreuses bibliothèques disponibles.

2. Langages et cadriciels



Twig

- Moteur de templates intégré à l'écosystème Symfony.
- Permet de séparer la logique PHP de la présentation HTML.
- Syntaxe simple et lisible.
- Évite de mélanger trop fortement HTML et logique applicative.
- Très adapté aux pages classiques du site.

React

- Permet de créer des interfaces dynamiques sous forme de composants réutilisables.
- Adapté aux interfaces avec beaucoup d'interactions utilisateur.
- Permet de faire évoluer facilement l'interface.
- Très utilisé dans le développement web, avec un écosystème important.
- Mais il faut faire attention à ne pas donner l'impression que React est obligatoire partout.

2. Langages et cadriciels

Technologie	Rôle	Justification principale
Symfony	Framework back-end	Structure, sécurité, maintenabilité
PHP	Langage serveur	Adapté au web et utilisé par Symfony
Twig	Génération des pages HTML	Intégration avec Symfony et séparation logique/présentation
React	Interface dynamique	Composants interactifs et réutilisables



3. Base de données

Modèle choisi : PostgreSQL

JUSTIFICATIONS DE NOTRE CHOIX :

Open Source

Solution libre et sans coût, robuste sur les besoins relationnels complexes.

Gestion de types avancés

Gère nativement des types comme JSON et les tableaux pour un très haut niveau de paramétrage.

Bon support des contraintes

Pour les contraintes d’intégrité et de transactions (facturations/règlements).

Mutualisation d'une base commune

Répond à l’exigence SaaS multi-société du cahier des charges, une seule base en isolant les données de chaque jardin.



3. Base de données

	PostgreSQL	MySQL
Open Source	✓	✓
Données JSON	JSONB, indexation GIN	JSON natif, indexation différente
Pour la sécurité multi-jardins	RLS native	Pas de RLS native équivalente
Contraintes complexes	Très flexibles	Certaines limitations
Types avancés	Tableaux, JSONB, types personnalisés	Choix plus restreint

Selon nous, PostgreSQL est plus adapté pour gérer les données évolutives et mutualisées des Jardins de Cocagne comparé à d'autres modèles comme MySQL par exemple.

4. Back office & Front office

Le cahier des charges distingue deux publics : l'équipe du Jardin (salariés, salariés en insertion, bénévoles) et les clients. Nous proposons deux interfaces web servies par la même application Symfony.

Elles partagent la même logique métier et la même base de données, mais sont séparées par les droits d'accès. Cela évite de maintenir deux applications, ce qui va dans le sens de la simplicité de reprise demandée par le client.

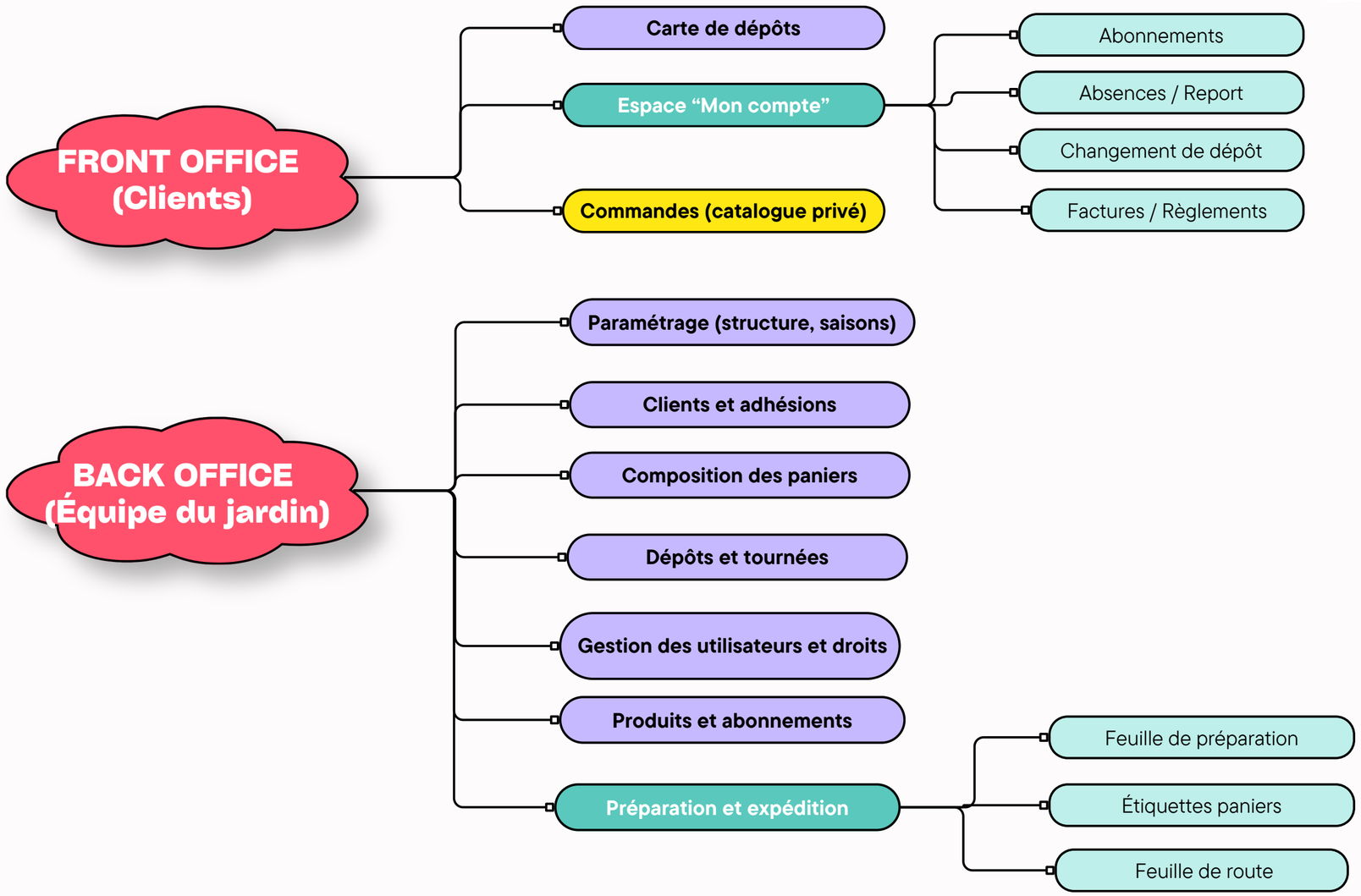
Back office : l'outil de gestion du Jardin

- Générer des interfaces de gestion paramétrables avec EasyAdmin (ou Sonata).
- Une interface épurée, des infobulles et une aide en ligne (tutoriels vidéo) pour aider ceux qui ne sont pas à l'aise avec l'informatique.
- Gestion des rôles et permissions (gestionnaire, bénévole, administrateur de la structure) répond au besoin de profils de droits du cahier des charges.
- Le très haut niveau de paramétrage exigé (saisons, fréquences, catégories de dépôts et de clients) est saisi ici, sans intervention d'un développeur.

Front office : l'espace client

- Pages publiques : carte des dépôts et présentation du Jardin.
- Espace « Mon compte » : abonnements, absences, changement de dépôt, factures et règlements. Le client devient autonome, ce qui décharge l'équipe.
- Rendu serveur en Twig, avec Vue.js réservé aux composants interactifs.
- Interface adaptée aux mobiles (responsive), car les clients consultent souvent depuis leur téléphone.
- Utilisation de la librairie React

4. Back office & Front office



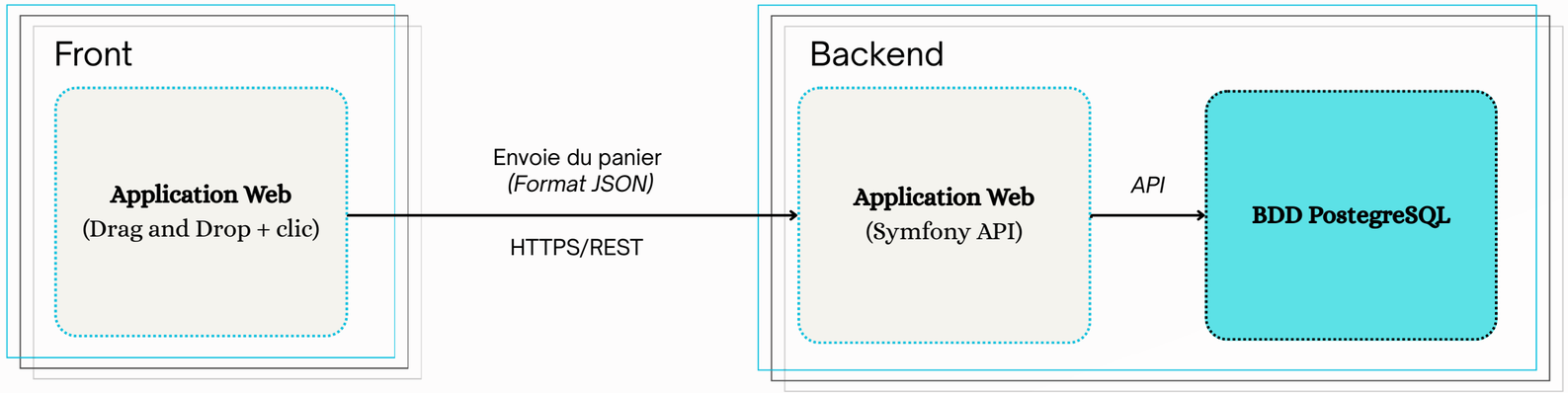
5. Quelques solutions techniques

a) Composition de panier

Afin d'optimiser la gestion des paniers, l'interface développée en React remplacera les formulaires HTML classiques par des outils plus instinctifs, facilitant ainsi la prise en main par l'équipe du Jardin et leur clients.

- Drag and Drop (et clic) : ajout et retrait des produits de manière intuitive.
- Calcul en instantané : mise à jour en temps réel du coût et de la composition du panier.
- Stockage en JSON : structuration flexible des données hétérogènes du panier.
- API interne Symfony : validation sécurisée des contraintes métiers (prix, stocks) avant l'enregistrement final sur PostgreSQL.





5. Quelques solutions techniques



b) IoT Surveillance de serre

Dans cette section nous allons voir les différents moyen existant pour faire communiquer les instruments avec l'application web.

MQTT (Message Queuing Telemetry Transport) → ESP32 + Raspberry Pi en tant que serveur
Avantage :
- Données en direct
- BDD intégrée au système

Désavantage :
- Nécessite la configuration d'un serveur intermédiaire (le broker sur le Raspberry Pi)

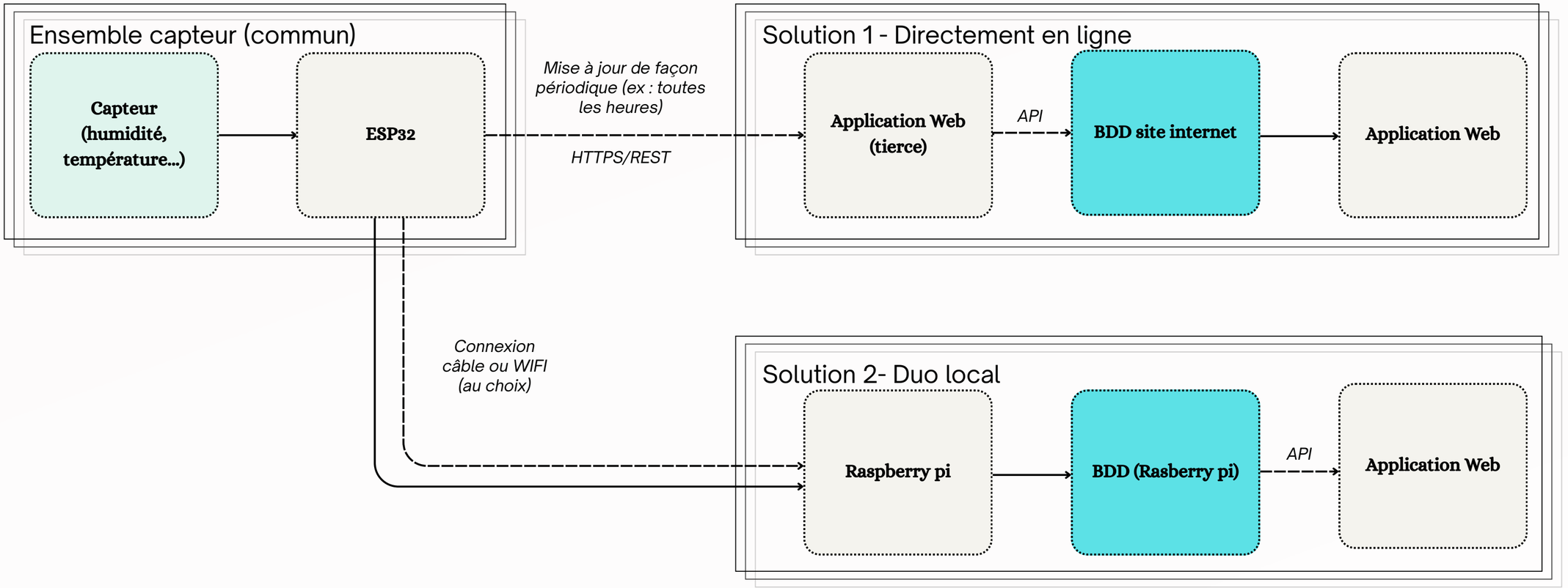
HTTP/REST → ESP32 uniquement
Avantage :
- Connexion directe et facile à l'API du site web

Désavantage :
- Donnée mise à jour périodiquement
- Ajout des valeurs à la BDD de l'app web



5. Quelques solutions techniques

b) IoT Surveillance de serre



5. Quelques solutions techniques



b) IoT Surveillance de serre

Afin d'éviter de monter un site internet tiers et son API nous allons partir vers la solution numéro 2.

La connexion entre l'ESP32 et le Raspberry pi se fera par câble directement.
La gestion de l'API se fera directement via le raspberry pi

Voici les avantages que cela apporte :

- Simplification pour les débutants en informatique ("Faut que le câble soit branché")
- Plus résilient, les ondes wifi peuvent être perturbées (structure métallique et humidité) pas un câble
- Peu onéreux, pas besoin d'un abonnement supplémentaire à un cloud pour héberger son 'deuxième' site internet
- Souveraineté des données, les données sont hébergées et restent gérées par les Jardins
- Evolution possible et 'facile' avec le projet de centralisation futur

ESP32
Langage :
- Arduino

Raspberry pi
Langage :
- Python

BDD → SQLite
API → FastAPI



5. Quelques solutions techniques

c) CALENDRIER



Chaque Jardin livre des paniers à des rythmes différents (chaque semaine, tous les 15 jours, une fois par mois) sur plusieurs tournées, avec des semaines de fermeture et des jours fériés qui décalent les livraisons.

Sans automatisation, l'équipe doit déterminer à la main, pour chaque semaine, quels abonnés doivent être livrés, ce qui serait long et source d'erreurs. Le calendrier est donc un moteur de calcul qui alimente directement la préparation des commandes et l'espace client.



PHP-RRULE + SERVICE SYMFONY

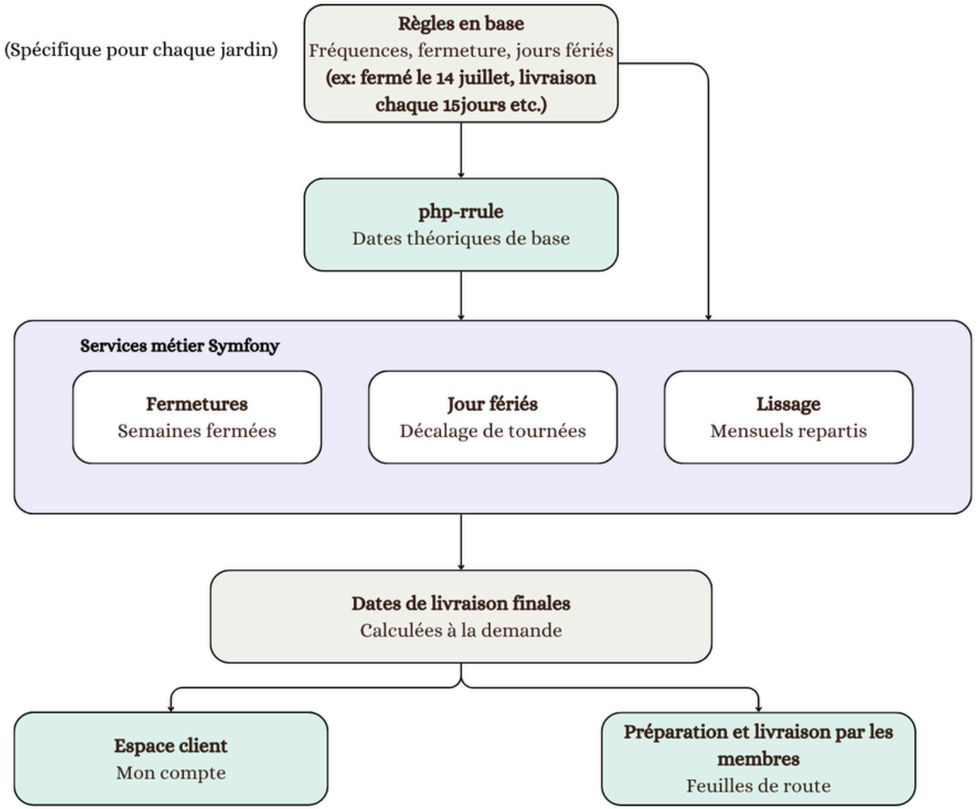
- Générer les dates de récurrence de base (hebdomadaire, tous les 15 jours, mensuel).
- Appliquer les règles propres à chaque Jardin : fermetures, décalages de jours fériés.
- Répartir les abonnements mensuels sur plusieurs semaines.
- Stocker en base les règles plutôt que des dates figées, ce qui permet à toute modification (nouvelle fermeture, jour férié) de se répercuter automatiquement.

5. Quelques solutions techniques

c) CALENDRIER



Fonctionnement:



5. Quelques solutions techniques

c) CALENDRIER



Exemple concret:



- Cliente “Marie”
- Panier tous les 15jrs
- Tournée du mardi

Étape 1 : php_rrules : Dates théoriques

Les jours de livraison de Marie sont:
mar 30 juin /mar 14 juil /mar 28 juil /mar 11 août

Étape 2 : Services métier Symfony : application des règles

- 30 juin → rien à signaler
- 14 juil → jour férié (règle de la tournée du mardi) ⇒ décalé au mercredi 15 juil
- 28 juil → semaine fermée (27 juil – 2 août) ⇒ livraison supprimée
- 11 août → rien à signaler

Étape 3 : Ce que Marie voit dans l’espace « Mon compte »

- mar 30 juin : livré
- mar 14 juil : livraison décalée le 15 juil (férié le 14 juil)
- mar 28 juil : pas de livraison (fermeture du Jardin)
- mar 11 août : livraison

5. Quelques solutions techniques

d) Carte et itinéraire

L'objectif est d'optimiser les tournées de livraison vers de multiples points de dépôt. L'enjeu majeur consiste à intégrer une interface cartographique pour localiser ces dépôts et la coupler à un moteur de calcul d'itinéraires pour générer des feuilles de route optimisées (et cohérentes).



Technologie	Avantage	Inconvenient
Google Maps Platform (Maps JavaScript API + Routes/Directions API)	• Qualité (précision) • Donnée de trafic en temps réel • Robuste	• Onéreux (si requête nombreuse) • Dépendance à google
OpenStreetMap + Leaflet (Front-office) et OSRM (Back-office)	• Léger • Facilement compatible avec les framework • Hébergement local (souveraineté des données)	• Peut manquer de précision • Lourd si la map est trop grande
HERE Technologies (Routing API + Waypoint Sequence API)	• Système historique • Prend en compte le gabarit du véhicule • Très performant pour les points d'arrêt multiple	• Apprentissage ardu • Coût de licence exorbitant

5. Quelques solutions techniques

d) Carte et itinéraire

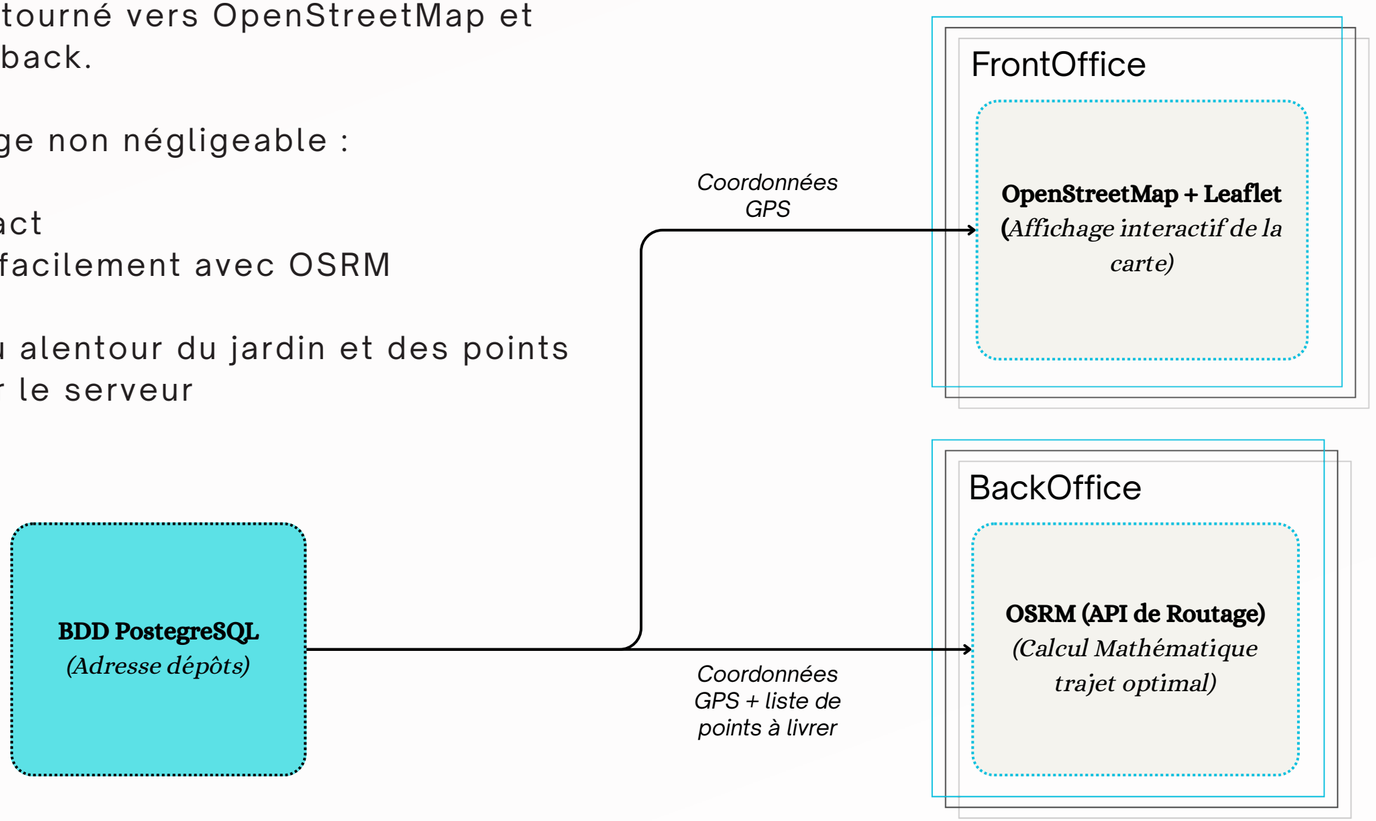
Notre choix pour la gestion de la carte et de l'itinéraire, afin que les clients puissent choisir leurs dépôts et que l'on puisse calculer les itinéraires de manière optimale, s'est tourné vers OpenStreetMap et Leaflet pour le front et OSRM pour le back.

Pour une association, c'est un avantage non négligeable :

- Open source et gratuit
- Légèreté et Compatibilité avec React
- Possibilité de gérer les itinéraires facilement avec OSRM

Nous limiterons la taille de la carte au alentour du jardin et des points de dépots afin de limiter l'impacts sur le serveur





Conclusion

En mettant en place un outil simple et adapté, on accompagne les Jardins de Cocagne dans leur transition numérique.

Ce document expose ainsi les solutions techniques et les choix architecturaux retenus pour le développement d'une application web robuste, scalable et ergonomique.

Nous souhaitons rendre la gestion des jardins plus aisée en proposant une solution numérique inclusive, adaptée aux utilisateurs de tous niveaux.

